

SENAI

Curso Técnico em Desenvolvimento de Sistemas

CASOS DE TESTE

Documento complementar ao Plano de Testes

Projeto de referência: VetCare API

Módulos cobertos: Autenticação, Consultas e Pagamentos

Este documento é um EXEMPLO. Use-o como modelo, substituindo cada caso pelos cenários reais do seu projeto.

Sobre este documento

Enquanto o Plano de Testes define a estratégia (o que testar, com que prioridade, quais critérios de entrada/saída), este documento detalha cada cenário individual que será executado, passo a passo. Cada caso de teste valida uma única condição de um endpoint.

Os casos abaixo cobrem os endpoints classificados como prioridade Alta no Plano de Testes da VetCare API (Autenticação, Consultas e Pagamentos) e materializam os riscos já identificados naquele documento — por exemplo, o caso TC-PAG-02 testa exatamente o risco de "cobrança duplicada por reenvio de requisição".

Campos de cada caso: Pré-condição (o que precisa estar preparado antes de executar), Passos (ação realizada), Dados de entrada (payload enviado), Resultado esperado (o que deveria acontecer), Resultado obtido (o que aconteceu de fato na execução) e Status.

Módulo: Autenticação

TC-AUTH-01 — Login com credenciais válidas

Pré-condição	Usuário cadastrado com e-mail e senha válidos, conta ativa.
Passos	1. Enviar POST /auth/login com e-mail e senha corretos. 2. Aguardar a resposta da API.
Dados de entrada	{ "email": "vet@vetcare.com", "senha": "SenhaValida123" }
Resultado esperado	Status 200. Corpo da resposta contém accessToken e refreshToken válidos.
Resultado obtido	Status 200 retornado, com os dois tokens presentes e válidos.
Status	Aprovado

TC-AUTH-02 — Login com senha incorreta

Pré-condição	Usuário cadastrado; senha enviada é diferente da senha correta.
Passos	1. Enviar POST /auth/login com e-mail correto e senha incorreta.
Dados de entrada	{ "email": "vet@vetcare.com", "senha": "SenhaErrada" }
Resultado esperado	Status 401 Unauthorized, com mensagem genérica de erro (sem indicar se o e-mail existe ou não).
Resultado obtido	Status 401 retornado com mensagem "Credenciais inválidas".
Status	Aprovado

TC-AUTH-03 — Login com e-mail inexistente

Pré-condição	Nenhum usuário cadastrado com o e-mail informado.
Passos	1. Enviar POST /auth/login com um e-mail que não existe na base.
Dados de entrada	{ "email": "naoexiste@vetcare.com", "senha": "Qualquer123" }
Resultado esperado	Status 401 Unauthorized, com a MESMA mensagem genérica do caso TC-AUTH-02 (não deve revelar quais e-mails estão cadastrados).
Resultado obtido	Status 401 retornado com a mesma mensagem "Credenciais inválidas".
Status	Aprovado

Módulo: Consultas

TC-CONS-01 — Agendar consulta com dados válidos

Pré-condição	Tutor autenticado; pet cadastrado e vinculado a esse tutor; veterinário com horário livre.
Passos	1. Autenticar como tutor e obter o accessToken. 2. Enviar POST /consultas com petId, veterinariold e dataHora válidos.
Dados de entrada	{ "petId": 12, "veterinariold": 3, "dataHora": "2026-09-10T14:00:00" }
Resultado esperado	Status 201 Created. Corpo retorna a consulta criada com status "agendada".
Resultado obtido	Status 201 retornado, consulta criada corretamente com status "agendada".
Status	Aprovado

TC-CONS-02 — Agendar em horário já ocupado (conflito)

Pré-condição	Já existe uma consulta confirmada para o mesmo veterinário, no mesmo horário do TC-CONS-01.
Passos	1. Repetir o agendamento com o mesmo veterinariold e a mesma dataHora usados no TC-CONS-01.
Dados de entrada	{ "petId": 20, "veterinariold": 3, "dataHora": "2026-09-10T14:00:00" }
Resultado esperado	Status 409 Conflict. Mensagem "Horário indisponível para este veterinário". Nenhuma segunda consulta é criada.
Resultado obtido	Status 409 retornado com a mensagem esperada; nenhuma consulta duplicada foi criada.
Status	Aprovado

TC-CONS-03 — Agendar consulta para pet de outro tutor (autorização)

Pré-condição	Tutor A autenticado; o petId informado pertence ao Tutor B.
Passos	1. Autenticar como Tutor A. 2. Enviar POST /consultas usando o petId de um pet do Tutor B.
Dados de entrada	{ "petId": 45, "veterinariold": 3, "dataHora": "2026-09-11T09:00:00" }
Resultado esperado	Status 403 Forbidden. Nenhuma consulta é criada.
Resultado obtido	Status 403 retornado corretamente; consulta não foi criada.
Status	Aprovado

TC-CONS-04 — Cancelar consulta existente

Pré-condição	Consulta previamente agendada (ex.: a criada no TC-CONS-01).
Passos	1. Enviar PATCH /consultas/:id/status com o novo status "cancelada".
Dados de entrada	{ "status": "cancelada" }
Resultado esperado	Status 200. Consulta passa a ter status "cancelada" e o horário é liberado na agenda do veterinário.
Resultado obtido	Status 200 retornado; horário voltou a aparecer como disponível em GET /veterinarios/:id/agenda.

Status	Aprovado
--------	----------

Módulo: Pagamentos

TC-PAG-01 — Cobrança de consulta com dados válidos	
Pré-condição	Consulta realizada; cartão de teste válido configurado no sandbox do gateway de pagamento.
Passos	1. Enviar POST /pagamentos com consultald, valor e método de pagamento.
Dados de entrada	{ "consultald": 45, "valor": 150.00, "metodo": "cartao_credito" }
Resultado esperado	Status 201 Created. Pagamento retornado com status "aprovado".
Resultado obtido	Status 201 retornado; pagamento aprovado no sandbox.
Status	Aprovado

TC-PAG-02 — Reenvio da mesma requisição de pagamento (idempotência)	
Pré-condição	Pagamento do caso TC-PAG-01 já processado com sucesso.
Passos	1. Reenviar exatamente a mesma requisição do TC-PAG-01 (mesmo consultald e mesma chave de idempotência).
Dados de entrada	{ "consultald": 45, "valor": 150.00, "metodo": "cartao_credito" }
Resultado esperado	Status 200. A API reconhece a chave de idempotência e retorna o pagamento já existente, SEM criar uma nova cobrança.
Resultado obtido	BUG ENCONTRADO: a API criou uma segunda cobrança de R\$ 150,00 para a mesma consulta — a chave de idempotência não está sendo validada no endpoint. Corresponde ao risco "cobrança duplicada" já previsto no Plano de Testes.
Status	Reprovado

TC-PAG-03 — Pagamento com cartão recusado (sandbox)	
Pré-condição	Cartão de teste do sandbox configurado para simular recusa.
Passos	1. Enviar POST /pagamentos usando o cartão de teste de recusa.
Dados de entrada	{ "consultald": 46, "valor": 150.00, "metodo": "cartao_credito" }
Resultado esperado	Status 402 Payment Required. Status do pagamento "recusado". A consulta permanece com status "agendada" (não é marcada como paga).
Resultado obtido	Status 402 retornado; pagamento marcado como "recusado"; consulta manteve status "agendada" corretamente.
Status	Aprovado

Resumo da execução

9 de 10 casos aprovados. 1 caso reprovado (TC-PAG-02), com bug crítico de cobrança duplicada — bloqueia o critério de saída definido no Plano de Testes ("nenhum bug de severidade Alta em aberto"), então este item deve ser corrigido e reexecutado antes da aprovação final do módulo de Pagamentos.

Como adaptar este modelo para o seu projeto

- Um caso de teste por cenário: sucesso, cada tipo de erro esperado e cada regra de negócio/autorização específica do seu sistema.
- Dê IDs previsíveis (ex.: TC-<MÓDULO>-<NÚMERO>) para facilitar referência no relatório de bugs e no código de teste automatizado.
- Sempre que possível, escreva o caso de teste ANTES de implementar o teste automatizado (.test.js) — o caso descrito em português é o que orienta o código.
- Preencha "Resultado obtido" só depois da execução real; antes disso, deixe como "Não executado".
- Um caso reprovado deve gerar um item no relatório de bugs, referenciando o ID do caso de teste que o encontrou.